

La librería Datetime en Python

Para manejar fechas en Python se suele utilizar la librería datetime que incorpora los tipos de datos date, time y datetime para representar fechas y funciones para manejarlas. Algunas de las operaciones más habituales que permite son:

- Acceder a los distintos componentes de una fecha (año, mes, día, hora, minutos, segundos y microsegundos).
- Convertir cadenas con formato de fecha en los tipos date, time o datetime.
- Convertir fechas de los tipos date, time o datetime en cadenas formateadas de acuerdo a diferentes formatos de fechas.
- Hacer aritmética de fechas (sumar o restar fechas).
- Comparar fechas.

Previamente hay que instalar en el terminal del equipo (CMD, Powershell o Linux como administrador) los siguientes comandos:

- `pip install datetime`
- `pip install python-dateutil`

Para importar la librería Datetime y sus componentes debemos adjuntar al principio del programa, función o procedimiento que usemos las siguientes líneas:

- Para trabajar con fechas en general, horas y días
`from datetime import date, datetime, timedelta`
- Para trabajar con fechas tipo años, meses, minutos y segundos
`from dateutil.relativedelta import relativedelta`

Los tipos de datos date, time y datetime

- **date(año, mes, día)** : Devuelve un objeto de tipo date que representa la fecha con el año, mes y día indicados.
- **time(hora, minutos, segundos, microsegundos)** : Devuelve un objeto de tipo time que representa un tiempo la hora, minutos, segundos y microsegundos indicados.
- **datetime(año, mes, día, hora, minutos, segundos, microsegundos)** : Devuelve un objeto de tipo datetime que representa una fecha y hora con el año, mes, día, hora, minutos, segundos y microsegundos indicados.

Acceso a los componentes de una fecha

- **date.today()**: Devuelve un objeto del tipo date la fecha del sistema en el momento en el que se ejecuta.
- **datetime.now()**: Devuelve un objeto del tipo datetime con la fecha y la hora del sistema en el momento exacto en el que se ejecuta.
- **d.year**: Devuelve el año de la fecha d, puede ser del tipo date o datetime.
- **d.month**: Devuelve el mes de la fecha d, que puede ser del tipo date o datetime.
- **d.day**: Devuelve el día de la fecha d, que puede ser del tipo date o datetime.
- **d.weekday()**: Devuelve el día de la semana de la fecha d, que puede ser del tipo date o datetime.
- **t.hour**: Devuelve las horas del tiempo t, que puede ser del tipo time o datetime.
- **t.minute**: Devuelve los minutos del tiempo t, que puede ser del tipo time o datetime.
- **t.second**: Devuelve los segundos del tiempo t, que puede ser del tipo time o datetime.
- **t.microsecond**: Devuelve los microsegundos del tiempo t, que puede ser del tipo time o datetime.

Ejemplos:

```
>>> print(date.today())    →    2023-04-11
>>> dt = datetime.now()
>>> dt.year                →    2023
>>> dt.month               →    4
>>> dt.day                 →    11
>>> dt.hour                →    22
>>> dt.minute              →    5
>>> dt.second              →    45
>>> dt.microsecond         →    1338
```

Conversión de fechas en cadenas con diferentes formatos

- **d.strftime(formato):** Devuelve la cadena que resulta de transformar la fecha `d` con el formato indicado en la cadena `formato`. La cadena `formato` puede contener los siguientes marcadores de posición:
 - `%Y` - El año con siglos, 2023.
 - `%y` - El año sin siglos, 23.
 - `%m` - El mes como un número acompañado de cero, como es el caso de 01.
 - `%B` - El nombre completo del mes, como January.
 - `%b` - El nombre abreviado del mes, como Jan.
 - `%d` - El día del mes con un número sin ceros, formato 8.
 - `%A` - El nombre completo del día de la semana, Sunday.
 - `%a` - El nombre abreviado del día de la semana, por ejemplo, domingo (Sunday) se abrevia a Sun
 - `%I` - Las horas del día en un formato de 12 horas, como 6 (PM).
 - `%H` - Las horas del día en un formato de 24 horas, como 18.
 - `%M` - Los minutos en una hora, en formato 20.
 - `%S` - Los segundos en un minuto, en formato 00.

- %p - Configuración AM o PM.
- %C - Muestra fecha y hora completas.
- %x - Muestra fecha completa.
- %X - Muestra hora completa.

Ejemplos:

```
>>> d = datetime.now()
>>> print(d.strftime('%d-%m-%Y'))      →      13-04-2020
>>> print(d.strftime('%A, %d %B, %y'))  →      Monday, 13 April, 20
>>> print(d.strftime('%H:%M:%S'))      →      20:55:53

>>> print(d.strftime('%H horas, %M minutos y %S segundos'))
                                          →20 horas, 55 minutos y 53 segundos
```

Conversión de cadenas en fechas

- **strptime(s, formato):** Devuelve el objeto de tipo date, time o datetime que resulta de convertir la cadena s de acuerdo al formato indicado en la cadena formato. La cadena formato puede contener los mismos marcadores de posición que en la función **d.strftime(formato)**

```
>>> datetime.strptime('15/4/2020', '%d/%m/%Y')
datetime.datetime(2020, 4, 15, 0, 0)

>>> datetime.strptime('2020-4-15 20:50:30', '%Y-%m-%d %H:%M:%S')
datetime.datetime(2020, 4, 15, 20, 50, 30)
```

Aritmética de fechas

Para representar el tiempo transcurrido entre dos fechas se utiliza el tipo `timedelta`.

- `timedelta(días, segundos, microsegundos)` : Devuelve un objeto del tipo `timedelta` que representa un intervalo de tiempo con los días, segundos y micorsegundos indicados.
- `d1 - d2` : Devuelve un objeto del tipo `timedelta` que representa el tiempo transcurrido entre las fechas `d1` y `d2` del tipo `datetime`.
- `d + delta` : Devuelve la fecha del tipo `datetime` que resulta de sumar a la fecha `d` el intervalo de tiempo `delta`, donde `delta` es del tipo `timedelta`.

```
>>> d1 = datetime(2020, 1, 1)
>>> d1 + timedelta(31, 3600)
datetime.datetime(2020, 2, 1, 1, 0)
>>> datetime.now() - d1
datetime.timedelta(days=132, seconds=1826, microseconds=895590)
```

Ejemplos de manipulación de fechas

Día actual

```
today = date.today()
```

Fecha actual

```
now = datetime.now()
```

Una vez obtengamos la fecha actual podremos obtener el día, mes, año, hora, minutos y segundos

Date

```
print("El día actual es {}".format(today.day))  
print("El mes actual es {}".format(today.month))  
print("El año actual es {}".format(today.year))
```

Datetime

```
print("El día actual es {}".format(now.day))  
print("El mes actual es {}".format(now.month))  
print("El año actual es {}".format(now.year))  
  
print("La hora actual es {}".format(now.hour))  
print("El minuto actual es {}".format(now.minute))  
print("El segundo actual es {}".format(now.second))
```

Conversión de una cadena a fecha

```
fecha = "31/12/2023 23:59:59"
```

En esta parte aplicamos la conversión y creación del objeto fecha

```
date_time_object = datetime.strptime(fecha, "%d/%m/%Y %H:%M:%S")
```

```
print(date_time_object)    → 2023-12-31 23:59:59
```

Para trabajar solamente con la fecha

```
date_object = datetime.strptime(fecha, "%d/%m/%Y %H:%M:%S").date()
```

```
print(date_object)          → 2023-12-31
```

Para trabajar solamente con la hora

```
time_object = datetime.strptime(fecha, "%d/%m/%Y %H:%M:%S").time()
```

```
print(time_object)          → 23:59:59
```

Conversión de una cadena a fecha solamente

```
fecha = "31/12/2023"
```

En esta parte aplicamos la conversión y creación del objeto fecha

```
date_time_object = datetime.strptime(fecha, "%d/%m/%Y")
```

```
print(date_time_object.day)  # Aquí obtenemos el día          → 31
```

```
print(date_time_object.month) # Aquí obtenemos el mes          → 12
```

```
print(date_time_object.year) # Aquí obtenemos el año           → 2023
```

Sumar días u horas a la fecha actual. Para restar usar números negativos

```
new_date = now + timedelta(days=2)          # Sumando 2 días
```

```
new_date = now + timedelta(hours=2)         # Sumando 2 horas
```

Sumar años, meses, minutos o segundos a la fecha actual. Para restar usar números negativos

```
new_date = now + relativedelta(years=2)    # Sumando 2 años
```

```
new_date = now + relativedelta(months=2)    # Sumando 2 meses
```

```
new_date = now + relativedelta(minutes=2)    # Sumando 2 minutos
```

```
new_date = now + relativedelta(seconds=2)    # Sumando 2 segundos
```

```
print(new_date)          → Fecha con los incrementos
```

Comparación de fechas

```
if now < new_date:  
    print("La fecha actual es menor que la nueva fecha")
```

Pregunta: ¿Cuántos días quedan para Fin de Año?

```
dias = (date_object - today).days
```

```
print ("Faltan " + str(dias) + " días para Fin de Año")
```